

REDCap record_id — test vs production

How a survey submission is attributed to a REDCap record. Korean mirror: `REDCAP_RECORD_ID_MAPPING_KR.md`. Source of truth: `app/Backend/redcap_mapping.py`, `deid.py`.

The idea (why a mapping)

A consultation transcript carries a **study SID** (e.g. `SID_22`). REDCap, however, keys its records by its **own auto-numbered record_id** (1, 2, 3 ...) assigned when a coordinator registers the patient. The two are different identifiers linked by a **mapping**. The backend must attribute each submission to REDCap's `record_id`, not invent one.

The single glue across the 4 systems (REDCap · pipeline · DB · dashboard) is that REDCap `record_id`; the SID is the study-side id that resolves to it.

Test version (current default) — `record_id == SID`

Because we know the `data/input` files, the test setup forces `record_id == SID`: the backend un-hashes the de-identified speaker to its SID (`deid.unhash_patient_sid`, e.g. `Patient_13511_...` → `SID_22`) and uses that verbatim as the REDCap `record_id`; `scripts/seed_redcap_record_ids.py` pre-creates `SID_21...` records. This is only valid when `record_id` and `SID` are deliberately made equal.

Production version — map SID → REDCap's auto-numbered record_id

Coordinator (REDCap):

- register patient → REDCap assigns `record_id` (1,2,3...)
- write the patient's SID into a REDCap field (`settings.redcap_sid_field`, e.g. `"study_sid"`)
- | → REDCap now holds the SID ↔ `record_id` link

Backend (runtime):

- read-only export of {`record_id`, <`sid_field`>} from REDCap → build {SID: `record_id`} (cached)
- survey submitted → un-hash speaker → SID
- `record_id = resolve_record_id(SID)` ← REDCap's own id, from the mapping
- if `record_id` is None (SID not registered) → DO NOT push; store the row as `pending` (`redcap_synced=false`, `redcap_error="SID ... not registered in REDCap"`)
- else push the answers to REDCap under that `record_id`; store it in `redcap_record_id`

- The DB `sid` column keeps the study `SID_<n>`; `redcap_record_id` now holds REDCap's **real** id.
- No seeding — REDCap owns the records.
- The mapping is cached (Redis, ~5 min TTL, with an in-process fallback) and **refreshed once on a miss**, so a just-registered patient resolves without waiting for the TTL.

Test vs production at a glance

	Test (current)	Production
<code>record_id</code>	the un-hashed SID (<code>SID_22</code>)	REDCap's auto number (<code>3</code>)
<code>source</code>	dashboard invents it	read from REDCap (<code>redcap_sid_field</code>)
<code>seeding</code>	<code>seed_redcap_record_ids.py</code> creates SID records	none (coordinator registers)
SID not in REDCap	n/a (we seeded it)	submission → pending (no push)
<code>redcap_record_id</code> stored	<code>SID_22</code>	<code>3</code>

Switching modes

The mode is a single config flag `REDCAP_RECORD_ID_MODE` (in `app/Backend/.env`): - **test** (default) — `record_id == SID`; keeps the current seeded-SID behavior working. - **production** — map SID → REDCap's auto id (needs the steps below).

Enabling production

- REDCap (operational)**: add a field holding the SID to every record (e.g. `study_sid`) and populate it when registering each patient. Without it, every submission resolves to *pending* (no data loss, no bad writes).
- Config**: set `REDCAP_RECORD_ID_MODE=production` and `REDCAP_SID_FIELD=<that field name>` in `app/Backend/.env` (SID field defaults

to `study_sid`).

3. Restart the backend. New submissions now resolve SID → REDCap record_id automatically.

Notes

- `unhash_patient_sid` still derives the SID from the hashed speaker; only the record_id source changed.
- Existing rows synced under the test scheme (`redcap_record_id` = `SID_...` / `Patient_...`) are stale under production; a one-time re-resolve script can update them (separate, writes to REDCap, run with care).
- The integrity checker (`integrity_checks.py` C2, `/admin/tracking/data-integrity`) reconciles each synced row against its stored `redcap_record_id` , so it works for both schemes.

Key files

- `app/Backend/redcap_mapping.py` — `load_sid_to_record_id` / `resolve_record_id` .
- `app/Backend/deid.py` — `unhash_patient_sid` (speaker → SID).
- `app/Backend/routes_surveys.py` , `routes_patient.py` — resolve at sync time.
- `app/Backend/core/settings.py` — `redcap_sid_field` .
- `app/Backend/scripts/seed_redcap_record_ids.py` — **test-only** seeder.